

Measuring the MCP Supply Chain: Eight Null Results and a Population That Is Not the Population

Prachet Poddar

University of California, Los Angeles

prachetpoddar@gmail.com

Version 1.0, 7 September 2026

This work was conducted independently. It was not funded by, supervised by, or carried out under the auspices of any research program, and it should not be attributed to the University of California.

Summary

Model Context Protocol servers are software that runs on developer machines, is launched by unpinned commands, and appears in no dependency manifest. That combination has attracted claims that the MCP ecosystem carries elevated supply-chain risk.

We measured it. Across 250 npm-distributed MCP servers, 2,868 of their transitive dependencies, 400 published packages, and 247 PyPI-distributed servers, we tested eight dimensions against coverage-matched, size-adjusted and age-adjusted controls: license validity, license text distribution, deprecated dependencies, release churn, abandonment, install-script execution, name collision, and license metadata completeness.

Seven came back null. On every dimension a package registry can expose, MCP servers on npm are statistically indistinguishable from comparable npm packages, and on two dimensions they are measurably better.

The eighth appeared to be a real finding and then dissolved under usage weighting. Python-distributed MCP servers are 2.4 times more likely than age-matched PyPI packages to ship with no license information at all, at 32.8% against a baseline near 18%. Weighted by actual installs, that rate falls to **1.9%**, a sixteen-fold collapse.

The reason is the most useful thing we measured. Downloads in this ecosystem are distributed with a **Gini coefficient of 0.96**, and a single package accounts for **63.6%** of all installs in our sample. The published population and the installed population are almost disjoint. Any study that samples packages uniformly, as every study of this kind including our own first seven rounds does, is describing software that essentially nobody runs. Docker's curated catalog, measured the same way, has a Gini of 0.71 and a top item at 11.4%, which suggests the tail is a property of open publication rather than of MCP.

We also report a sampling error that produced a spectacular false positive in our own data, and which we believe affects other work in this area.

1. Why this was measured

An MCP server is typically launched by a line in a client configuration file:

```
{ "command": "npx", "args": ["-y", "@scope/server"] }
```

Three properties follow, and we verified each directly rather than inferring them.

The configuration records no provenance. We retrieved the official `server.json` schema from the MCP registry and searched it. It contains zero occurrences of `license` or `spdx`. The `ServerDetail` object has eleven properties and requires three: `name`, `description`, `version`. The `Package` object carries an optional `version` and an optional `fileSha256`, and neither is required. The client configuration that actually launches the server discards even those.

Nothing is pinned. `npx -y <name>` re-resolves the latest distribution tag on every invocation, into a cache outside the project directory, recorded in no lockfile.

No dependency scanner sees any of it. A manifest-scanning tool pointed at a repository containing such a configuration returns zero components for the entire MCP surface.

To that we add a measured fact. Resolving launch commands and walking the resulting graphs, the median MCP server installs **97 packages** (mean 88.6, p90 165, maximum 589) at depth up to 10. A quarter install exactly one. Two thirds install more than 50.

The blind spot is real and its contents are numerous. This study asks a narrower question: **is the software in the blind spot unusually bad?**

2. Method

2.1 Sampling frames

npm. The registry search API reports 8,229 packages carrying the keyword `mcp-server`. Pagination stops near 5,000 results, giving a frame of 5,250 unique packages, or 64% coverage. Because search orders by score, this frame is a top-slice and our npm estimates are optimistic. We tested for a hygiene gradient across score quartiles and found none (point-biserial $r = -0.006$ for unevaluable licenses, $r = -0.059$ for missing license files, against a significance threshold of 0.098 at $n = 400$).

PyPI. The complete PyPI simple index is a single request returning **886,346 projects**. Filtering yields 18,963 packages carrying an `mcp` token and 4,062 that also contain `"server"`. Treatment and control arms are simple random samples from this complete enumeration, with no score ordering.

Controls. After an initial control failed (Section 6), we used coverage-matched npm frames:

Frame	Population	Our frame	Coverage
keywords:mcp-server	8,229	5,250	64%

Frame	Population	Our frame	Coverage
keywords:eslint-plugin	7,075	5,250	74%
keywords:langchain	1,973	1,973	100%

2.2 Dependency resolution

We resolved npm semantic-version ranges using an implementation of npm range semantics rather than resolving every edge to latest. Distribution tags are consulted first, then maximum-satisfying resolution over published versions. Optional dependencies are included; peer dependencies excluded. Non-registry specifiers are classified rather than dropped.

Resolution correctness matters. Re-walking 60 servers with correct range resolution against naive latest resolution changed the node count on **72% of them**, with a net increase of 15.5% in total package instances.

2.3 Usage weighting

Download counts come from the last-month PyPI statistics endpoint, retrieved for 221 of the 247 sampled packages; 26 returned no record.

2.4 Statistics

Proportions carry 95% normal-approximation intervals; comparisons use a two-proportion z test. Where an outcome scales mechanically with dependency-tree size or package age, we report Mantel-Haenszel odds ratios stratified on that confounder and restrict both arms to a common range.

3. Result: extreme usage concentration

This is reported first because it governs the interpretation of everything else.

Across 221 PyPI-distributed MCP servers with 203,446 combined monthly downloads:

Measure	Value
Gini coefficient	0.9611
Share taken by the single top package	63.6%
Share taken by the top 5	88.7%
Share taken by the top 10	93.0%
Share taken by the top 25	96.5%
Median downloads, top decile	687/month
Median downloads, bottom decile	8/month

One package accounts for roughly 129,000 of 203,000 monthly installs. The bottom half of the sample is measured in single-digit downloads per month.

An ecosystem of several thousand published packages is, in practice, a few dozen packages. Uniform sampling over the published population, which is what every measurement in Sections 4 and 5 does, describes software that is published but not used.

3.1 The curated channel has a different shape

Docker distributes MCP servers through a hand-maintained catalog rather than an open registry. We took all 328 entries from that catalog and fetched pull counts for each repository individually. 182 have a published image; the remaining 146 are catalogued but were never published.

Measure	PyPI (open registry)	Docker mcp/ (curated catalog)
Packages with usage data	221	182
Gini coefficient	0.9611	0.7094
Top 1 share	63.6%	11.4%
Top 5 share	88.7%	38.6%
Top 10 share	93.0%	53.3%
Top 25 share	96.5%	69.9%
Items with zero usage	bottom decile at 8/month	0

The two figures are not comparable in absolute terms. Docker reports lifetime pulls, PyPI reports one month, and both include CI and mirror traffic. Only the shape is comparable, and the shape differs. The curated channel is skewed but not degenerate: its top item takes 11.4% rather than 63.6%, its median image has 33,892 pulls, and no image in it has zero.

This is what curation does to a distribution. It is not evidence that Docker's servers are better; the catalog was assembled by selecting servers people already wanted. The point is that the long tail of unused packages, which is what dissolved the Section 5 finding, is largely a property of open publication rather than of MCP. A channel that publishes 328 things by hand does not have one.

It also gives the allowlist argument in Section 9 a concrete size. Docker has already built such a list, and it is 328 names long, of which 25 carry 70% of the pulls.

4. Result: seven nulls

#	Dimension	MCP	Control	z	Adjusted OR
1	Ships no	22.2% $\pm$ 4.1	26.0% $\pm$ 5.4	-1.09	

#	Dimension	MCP	Control	z	Adjusted OR
	license file	(n=400)			
2	Declares a license, ships no text	21.2% $\pm$ 4.0	24.8% $\pm$ 5.4	-1.05	0.82
3	Deprecated package in tree	17.2% $\pm$ 4.7 (n=250)	22.8% $\pm$ 5.2	-1.57	0.50
4	Released in last 30 days	28.5% $\pm$ 4.4	44.4% $\pm$ 6.2	-4.15	
5	Published exactly one version	25.2% $\pm$ 4.3	21.2% / 26.4%	+1.18 / -0.33	1.09
6	Install hook in tree	15.6% $\pm$ 4.5	14.2% $\pm$ 6.2	+0.36	0.79
7	Name collision	14.1% $\pm$ 0.9	20.7% / 9.1%	—	—

Three deserve comment.

Deprecated dependencies (row 3). A raw rate of 17.2% looks alarming alone. Against a shape-matched control it is lower than baseline, and stratified on tree size the odds ratio is 0.50. Deprecation exposure is a function of how many packages you install.

Release churn (row 4). Significant in the opposite direction. MCP servers publish less often than comparable npm applications.

Install scripts (row 6). The dimension we most expected to differ, because `npm -y` executes `preinstall`, `install` and `postinstall` for the root and every dependency. The absolute exposure is real: **15.6% of MCP server installations execute third-party code at install time, rising to 57.7% once the tree exceeds 120 packages**, usually through native-compilation packages such as `protobufjs`, `sharp`, `better-sqlite3` and `onnxruntime-node`. But `langchain` packages show 14.2% and the size-adjusted odds ratio is 0.79.

4.1 Why the nulls are consistent

License metadata on npm is saturated. Within 15 functional clusters the clean-SPDX rate runs 85% to 100% in both head and tail. Pooled, the 20 highest-ranked packages per cluster are 93.7% clean against a random tail at 98.7% ($z = -3.19$), so popularity selects weakly *against* license quality.

A variable sitting at 95% regardless of slicing cannot produce a differential result. `npm init -y` writes "license": "ISC" into every scaffolded package with no author decision, which sets the ceiling. It writes no LICENSE file, which is the direct cause of the roughly one-in-four rate at which npm packages of all kinds declare a license they do not ship.

5. Result: a finding that does not survive usage weighting

Python-distributed MCP servers are considerably more likely than the PyPI baseline to carry no license information in any of license, license_expression, or a license classifier.

	MCP servers	Control	
n	247	245	
No license information at all	32.8% ±5.9	18.0% ±4.8	z = +3.78
Median age since first upload	312 days	1,123 days	
Age-adjusted odds ratio			2.45
Both arms restricted to ≤600 days	32.7% ±5.9	14.5% ±7.9 (n=76)	z = +3.07

It survives age matching, the confound that destroyed two of our npm results. Our general-PyPI control rate of 18.0% agrees within its interval with the 15.41% reported for PyPI by a 2024 study of 33.7 million versioned packages, so the control measures what independent work says it should.

Then it collapses under usage weighting.

	Rate
Package-weighted	31.7% (95% CI 25.9 to 38.1), 70/221
Download-weighted	1.9% , 3,941 of 203,446 installs
Collapse factor	16.4x

The decile breakdown shows why. Missing licenses are concentrated almost entirely in packages nobody installs:

Decile	Median downloads/month	No license info
1 (most downloaded)	687	18.2%
2	175	4.5%
3	63	4.5%
4 to 6	38 to 18	22.7%
7	14	36.4%

Decile	Median downloads/month	No license info
8	13	45.5%
9 and 10	11 and 8	68.2%

Top three deciles average 9.1%; bottom four average 54.6%.

A prediction we got wrong, and it matters. Before obtaining download data we used release count and recency as adoption proxies. Packages with four or more releases and activity in the past year showed 31.1% missing, essentially identical to the overall rate, and we predicted the download-weighted figure would land between 20% and 35%. It is 1.9%. **Maintenance activity is not a proxy for adoption.** A package can publish ten releases and be installed eight times a month.

The finding is therefore true and unimportant: real for the published population, close to absent for the installed one.

6. A sampling error that produces false positives

Our first control set produced a spectacular and entirely false result.

Comparing MCP servers against npm packages keyworded `cli` or `server`, single-version publication appeared at 25.2% for MCP against 3.2% for the control, $z = +7.32$. Age was not the explanation: restricting both arms to packages under 561 days old preserved the effect at an age-adjusted odds ratio of 16.86.

The explanation was coverage. `keywords:cli` returns 104,818 packages and we had sampled the top 2,500 by search score, the top **2.4%**. Our MCP frame was 64% of a much smaller population. We had compared most of a small ecosystem against the elite of a large one.

Against coverage-matched frames the effect vanishes: `eslint-plugin` 26.4%, `langchain` 21.2%, MCP 25.2%, age-adjusted odds ratios 1.84 and 1.09. **Roughly a quarter of npm packages publish exactly once. That is what npm looks like when you do not select on popularity.**

Undetected, this error invalidated four earlier results before we caught it. Any study drawing an npm frame from the search API inherits it. We recommend matching coverage fractions explicitly across arms, or drawing frames from a source that enumerates completely, as the PyPI simple index does.

7. Incidental result: functional redundancy

Classifying all 5,250 frame packages by declared function:

Cluster	Servers	Cluster	Servers
memory/knowledge	302	browser automation	124
github/git	297	maps/weather	97
database/sql	293	slack/chat	90
finance/crypto	270	notion/docs	87
aws/cloud	163	web search	77
web fetch/scrape	135	filesystem	76
email	130	time/date	44
		jira/issues	42

Heavy redundancy: 297 packages address GitHub, 293 address databases, 76 address the filesystem. **Thirteen packages are effectively "the MySQL MCP server"**, one unscoped as `mcp-server-mysql`, which reads as canonical. Across collision groups, 126 of 274 contain an unscoped name and 62% show a rank gap greater than 1,500 places between their most and least prominent member. That is the shape typosquatting exploits, though the collision rate itself sits between langchain's 20.7% and eslint-plugin's 9.1%.

13 of 15 clusters have a cleanly licensed option among their five most prominent members, and the remaining two have three of five.

8. Limitations

The npm frame covers 64% of its population and is score-ordered. We found no hygiene gradient within it but cannot rule out that the excluded portion differs categorically.

Download data was obtained for PyPI only. The npm arm remains package-weighted, though Section 4.1 shows license quality does not vary with popularity there, so weighting would move those results little.

The download sample is 221 packages from a 4,062-package frame at one point in time, and last-month counts include mirror and CI traffic, which inflates absolute figures without obviously biasing the comparison.

Peer dependencies are excluded, so trees are undercounts. Docker-distributed servers are measured for usage concentration only (Section 3.1); their images were not inspected for license or maintenance metadata, so they contribute to no result in Sections 4 or 5.

We measured license and maintenance metadata. We did not measure malware, typosquatting outcomes, prompt injection, tool poisoning, or runtime capability scope, which is where most published MCP security work sits. **Our nulls say nothing about those dimensions.**

9. What this means

For anyone assessing MCP supply-chain risk on the dimensions a package registry exposes: this ecosystem is ordinary. Claims that it is unusually poorly licensed, unusually abandoned, or unusually prone to install-time code execution are not supported once matched controls are applied. The one dimension where a real difference exists affects 1.9% of installs.

The larger methodological point is that **the published population is the wrong unit of analysis**. With a Gini of 0.96 and a single package taking 63.6% of installs, per-package rates describe a long tail that is measured but not run. This applies to our own seven nulls as much as to the finding it dissolved, and it applies to any registry-scale study that samples uniformly.

There is a practical consequence. If a few dozen servers account for nearly all real usage, the governance problem is far smaller than package counts suggest, and a curated allowlist covering the top few dozen would address most deployments.

The structural facts stand independently of all of this. Agent tool configurations record no provenance, nothing is pinned, a median of 97 packages executes per server with 15.6% running install-time code, and no existing scanner sees any of it. Whether that absence of visibility is a problem anyone needs solved is not a question measurement can answer.

10. Data and code

Availability. Every measurement, sampling frame, control arm and analysis script is released at:

<https://github.com/prachetpoddar/mcp-supply-chain-study>
Archived copy with a citable DOI: 10.5281/zenodo.22641813
<https://doi.org/10.5281/zenodo.22641813>

The repository contains eleven analysis scripts, the resolver library, and fourteen JSON datasets covering all four rounds of measurement, including the two runs that were later found invalid and the corrections applied to them. Nothing has been removed to make the result look cleaner.

Licensing. Code is released under Apache-2.0. Data is released under CC-BY-4.0. The datasets consist of factual metadata retrieved from public registry APIs, together with derived statistics; no third-party package source is redistributed.

Provenance. The resolver uses only the public npm and PyPI registry APIs and the public Docker Hub repository API. Package trees were resolved with npm semver range semantics rather than by taking latest, which is why the round-one dataset is superseded. License detection uses the SPDX License List, 739 identifiers at time of measurement. Docker pull counts were taken one repository at a time from the 328-entry docker/mcp-registry catalog, because the namespace listing endpoint silently ignores its ordering parameter (Section 6).

Reproduction notes. Three things will otherwise reproduce our mistakes rather than our results.

1. Do not build a sampling frame from the npm search API without matching coverage fractions across arms. Section 6 gives the false positive this produces, $z = 7.32$ for an effect that does not exist.
2. Verify that an ordering parameter took effect before building anything on the result. Docker Hub accepts `ordering=-pull_count` and ignores it.
3. Do not skip a request that returns HTTP 429. Rate limiting is correlated with alphabetical position, which is correlated with nothing, until it is: our first Docker run lost the entire s-to-z tail, including six of the ten largest repositories.

Weight by usage before drawing conclusions about a population. That is the one instruction in this paper we would keep if we could keep only one.